

# CS-405: Deep Learning

## Generative Adversarial Networks - GANs

Dr. Sajjad Hussain

SEECS, NUST

March 31, 2026

# Lecture Outline

- 1 The Big Picture: Why GANs?
- 2 GAN Architecture
- 3 How GANs Work
- 4 Deep Convolutional GANs
- 5 Challenges in Training GANs
- 6 Wasserstein GANs
- 7 Conditional GANs

# Beyond VAEs: The Need for GANs

## VAE Limitations:

- Blurry generations
- Pixel-by-pixel loss doesn't capture global structure
- Tends to average possibilities

**Key Insight:** What if we let a network learn what "realistic" means, instead of defining it mathematically?

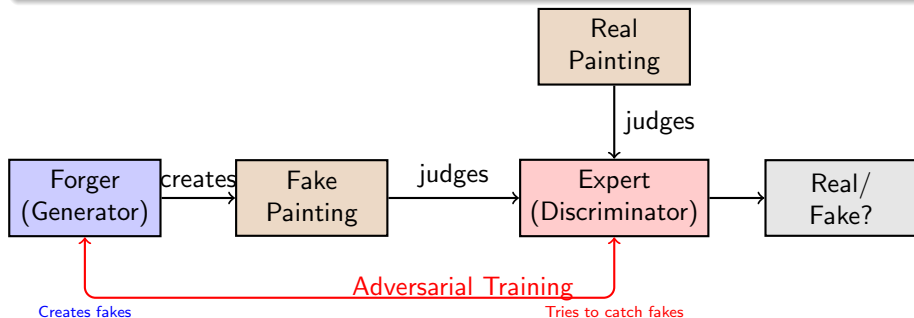
VAEs      Sharpness  $\rightarrow$       GANs

---

# The Core Idea: Learning by Competing

## Analogy: The Art Forger and the Art Expert

- **Forger (Generator):** Tries to create fake paintings that look real
- **Expert (Discriminator):** Tries to distinguish real from fake
- **Outcome:** Forger gets so good that expert can't tell the difference

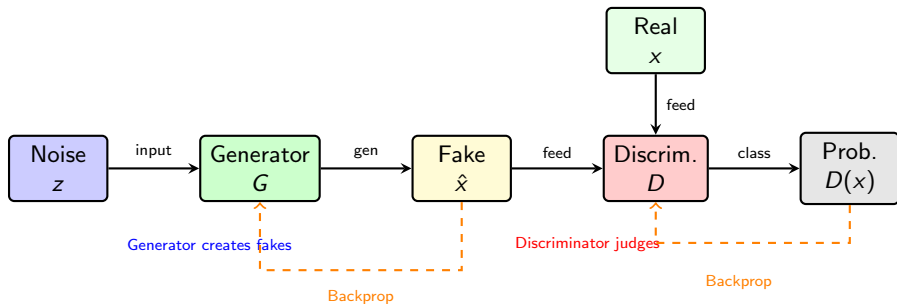


# The GAN Philosophy

## The Adversarial Game

- **Generator** improves by creating more convincing fakes
- **Discriminator** improves by becoming better at detection
- Both networks get better through competition

# Basic GAN Architecture



## Generator $G$

- Input: Random noise  $z$
- Output: Fake image  $\hat{x} = G(z)$
- Goal: Fool the discriminator

## Discriminator $D$

- Input: Real or fake image
- Output: Probability  $D(x) \in [0, 1]$
- Goal: Correctly identify real vs fake

# The Training Game: Minimax Objective

## The GAN Value Function

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

**Breakdown:**  
**Discriminator wants to:**

$$\max_D [\log D(x) + \log(1 - D(G(z)))]$$

- Make  $D(x)$  close to 1 for real
- Make  $D(G(z))$  close to 0 for fake

**Generator wants to:**

$$\min_G \log(1 - D(G(z)))$$

- Make  $D(G(z))$  close to 1
- Fool the discriminator

# The GAN Training Objective

## The Minimax Game

GAN training is a **two-player game** where:

- **Generator (G)**: Tries to create realistic fake data
- **Discriminator (D)**: Tries to distinguish real from fake
- Both improve by competing against each other



Each tries to outsmart the other!

## The Mathematical Formulation

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$



# Understanding the Notation

## What do the symbols mean?

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

### Key Symbols:

- $G$ : Generator network
- $D$ : Discriminator network
- $V(D, G)$ : Value function (what we optimize)
- $\min_G$ : Minimize with respect to  $G$
- $\max_D$ : Maximize with respect to  $D$
- $\mathbb{E}$ : Expectation (Averaged)

### Data Sources:

- $x \sim p_{\text{data}}$ : Real images from dataset
- $z \sim p_z$ : Random noise (e.g., Gaussian)
- $G(z)$ : Fake image from generator
- $D(\cdot)$ : Discriminator output (probability)

# The Discriminator's Objective

## Discriminator Maximizes

$$\max_D [\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]]$$

### Term 1: Real Images

$$\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]$$

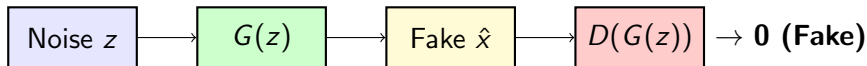
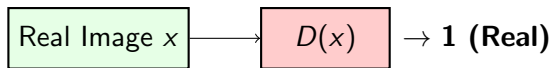
- $D(x)$ : Probability that  $x$  is real
- Want  $D(x) \approx 1$  for real images
- $\log(1) = 0$  (maximum value)
- $\log(0.5) = -0.69$  (penalty)

### Term 2: Fake Images

$$\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

- $D(G(z))$ : Probability fake image is real
- Want  $D(G(z)) \approx 0$  for fake images
- Then  $1 - D(G(z)) \approx 1$
- $\log(1) = 0$  (maximum value)

# What the Discriminator Wants



## Discriminator's Ideal Output

$$D(x) = \begin{cases} 1 & \text{if } x \text{ is real} \\ 0 & \text{if } x \text{ is fake} \end{cases}$$

Real → Fake Perfect discrimination!

# The Generator's Objective

## Generator Minimizes

$$\min_G \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

### What the Generator wants:

- Make  $D(G(z)) \approx 1$
- Then  $1 - D(G(z)) \approx 0$
- $\log(1) = 0$  (minimum value)
- $\log(0.5) = -0.69$  (worse)
- $\log(0.01) = -4.6$  (much worse!)

### Practical Problem:

- When  $D$  is very good,  
 $D(G(z)) \approx 0$
- Then  $\log(1 - 0) = \log(1) = 0$
- Gradient vanishes! (no learning)

# The Minimax Game: $\min_G \max_D$

What does  $\min_G \max_D$  mean?

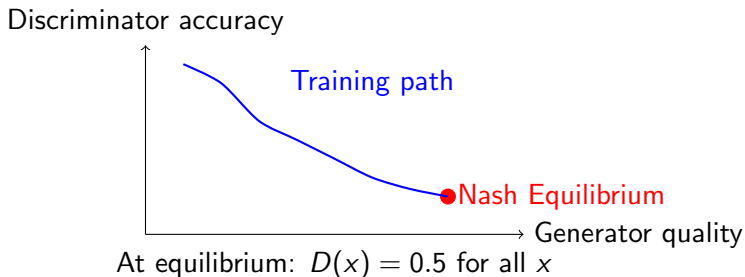
$$\min_G \max_D V(D, G)$$

- **Inner**  $\max_D$ : Discriminator chooses parameters to maximize  $V$
- **Outer**  $\min_G$ : Generator chooses parameters to minimize  $V$
- **Order matters!** Discriminator moves first (conceptually)

## Zero-Sum Game:

- What  $G$  gains,  $D$  loses
- $V(D, G)$  is like a score
- $D$  wants high score
- $G$  wants low score
- Both improve over time

# Visualizing the Minimax Game



## Key Insights

- Early training:  $D$  easily beats  $G$
- As training progresses: Both improve
- At equilibrium:  $G$  generates perfect fakes,  $D$  guesses randomly (50% accuracy)
- This is the **Nash Equilibrium** of the game

# Practical Training: Alternating Updates

For Each Training Iteration:

## 1 Update Discriminator:

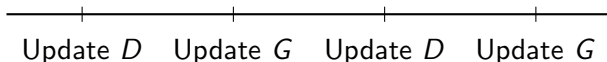
- Sample batch of real images  $\{x^{(1)}, \dots, x^{(m)}\}$
- Sample batch of noise  $\{z^{(1)}, \dots, z^{(m)}\}$
- Generate fake images  $\{G(z^{(1)}), \dots, G(z^{(m)})\}$
- Update  $D$  to maximize

$$\frac{1}{m} \sum_{i=1}^m [\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$$

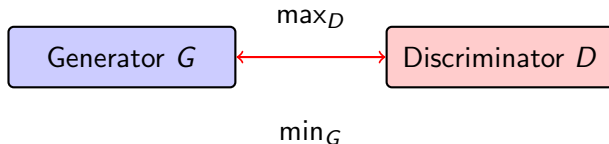
## 2 Update Generator:

- Sample new batch of noise  $\{z^{(1)}, \dots, z^{(m)}\}$
- Update  $G$  to minimize:  $\frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z^{(i)})))$

Alternating updates



# Summary: The GAN Value Function



## The Complete Objective

$$\min_G \max_D \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)]}_{\text{Real images: } D \text{ wants } 1} + \underbrace{\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]}_{\text{Fake images: } D \text{ wants } 0, G \text{ wants } 1}$$

### Discriminator's Job:

- Maximize both terms
- Push  $D(x) \rightarrow 1$
- Push  $D(G(z)) \rightarrow 0$

### Generator's Job:

- Minimize the second term
- Push  $D(G(z)) \rightarrow 1$
- Fool the discriminator



# Limitations of Basic GANs for Images

## Problem: MLP-based GANs struggle with images

- Use fully connected layers (MLPs)
- Ignore spatial structure of images
- Treat pixels as independent features
- Result: Unstable training, poor quality

## DCGAN Solution

Replace MLPs with Convolutional Neural Networks (CNNs)

- Preserve spatial information
- Learn hierarchical features (edges  $\rightarrow$  shapes  $\rightarrow$  objects)
- Much more stable training
- Higher quality generations

# DCGAN: Key Architectural Guidelines

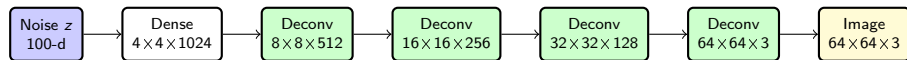
## Radford et al. (2015) introduced 5 key changes

- 1 Replace pooling with strided convolutions (discriminator)
- 2 Replace pooling with transposed convolutions (generator)
- 3 Use Batch Normalization in both networks
- 4 Remove fully connected hidden layers
- 5 Use specific activation functions

## Why these changes work

- Convolutions learn spatial hierarchies naturally
- Batch Normalization stabilizes training
- Removing FC layers reduces overfitting
- Proper activations prevent vanishing gradients

# DCGAN Generator Architecture



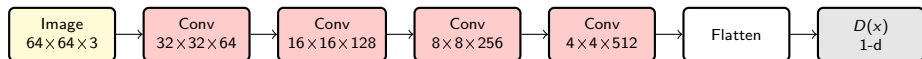
## Key Components:

- Starts with 100-d noise
- Dense layer projects to  $4 \times 4 \times 1024$
- Reshape to feature maps
- **Transposed convolutions** upsample
- Each layer: doubles size, halves channels

## Activations:

- **ReLU** in all layers (except output)
- **Tanh** in output layer ( $[-1,1]$  range)
- **BatchNorm** after each layer
- No pooling layers!

# DCGAN Discriminator Architecture



## Key Components:

- Takes  $64 \times 64 \times 3$  image input
- **Strided convolutions** downsample
- Each layer: halves size, doubles channels
- Ends with single probability output

## Activations:

- **LeakyReLU** in all layers ( $\alpha=0.2$ )
- **Sigmoid** in final output
- **BatchNorm** after each layer
- No pooling layers!

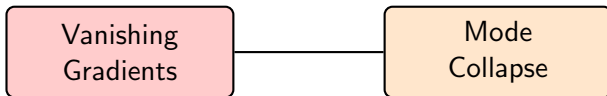
## What DCGAN Achieved (2015):

- Photorealistic faces ( $64 \times 64$ )
- Object generation (bedrooms, churches)
- Vector arithmetic in latent space
- Learned feature representations

## Legacy

- Became the standard architecture for GANs
- Enabled stable training of deeper GANs
- Paved the way for StyleGAN, BigGAN, etc.
- Still relevant as a baseline today

# Two Major Problems in GAN Training



## Vanishing Gradients

- Generator stops learning
- Discriminator becomes too good
- No useful gradient signal
- Training stagnates

## Mode Collapse

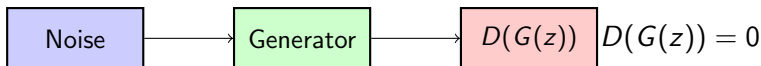
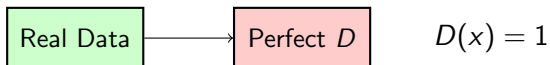
- Generator produces same output
- No diversity
- Collapses to few modes
- "Creative bankruptcy"

Wasserstein Loss solves both!

Vanishing Gradients      Mode Collapse

→

# Why Gradients Vanish: A Visual Explanation



What happens when  $D(G(z)) \approx 0$ ?

$$\text{Generator loss} = \log(1 - D(G(z))) \approx \log(1 - 0) = \log(1) = 0$$

- Loss is already at minimum
- Gradient = 0 (derivative of constant)
- Generator gets **no feedback** on how to improve. **Training Stops!**

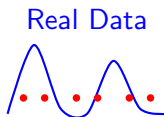
Generator stops learning → Training fails

# Mode Collapse: The Diversity Problem

## What is Mode Collapse?

Generator finds a **single "trick"** that fools the discriminator and keeps producing it

### Desired: Full Diversity



Generator covers all modes



### Mode Collapse

Generator stuck in one mode

Same face

Same face

Same face

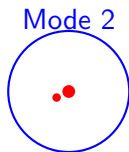
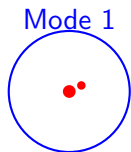
Generator produces identical outputs



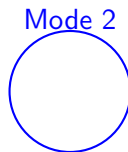
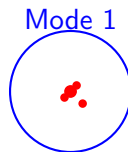
# Why Does Mode Collapse Occur?

## The Problem with Standard GAN Loss

Generator only needs to **fool the discriminator**, not cover the entire distribution



Good: Both modes covered



Bad: Only Mode 1 covered

## Analogy: The Lazy Student

- Teacher (discriminator) tests students
- Student (generator) finds one trick to pass
- Never learns the full subject material
- Gets stuck in a local optimum

# Wasserstein GAN (WGAN)

## Key Insight

Replace Log Loss with **Earth Mover's Distance (Wasserstein-1 Distance)**

### Standard GAN Loss:

$$\mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

- Log Loss
- Saturates when distributions don't overlap
- Vanishing gradients

### WGAN Loss:

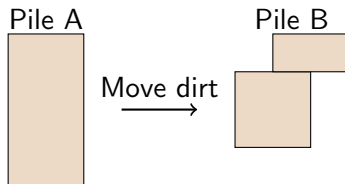
$$\mathbb{E}[D(x)] - \mathbb{E}[D(G(z))]$$

- Wasserstein Distance
- Smooth gradient everywhere
- No saturation

# Earth Mover's Distance (Wasserstein-1)

## The Analogy: Moving Dirt

Imagine you have two piles of dirt (distributions). The Earth Mover's Distance = **minimum work needed** to transform one pile into the other.



## Why This Helps GANs

- Log Loss with improved Discriminator → No gradient
- Wasserstein: **Continuous measure** of distance → Always provides gradient
- Even when distributions don't overlap, WGAN knows **how far apart** they are

# WGAN Loss Function: Overview

## Standard GAN vs WGAN

- **Standard GAN:** Discriminator outputs probability (real/fake)
- **WGAN:** Critic outputs a score (how "real" the image looks)

Standard GAN

Discriminator  
 $D(x) \in [0, 1]$



WGAN

Critic  
 $C(x) \in \mathbb{R}$

## WGAN Value Function

$$\min_G \max_C \underbrace{\mathbb{E}_{x \sim p_{\text{data}}} [C(x)]}_{\text{Real images score}} - \underbrace{\mathbb{E}_{z \sim p_z} [C(G(z))]}_{\text{Fake images score}}$$

- $C(x)$ : Critic network (outputs a real number, not probability)
- **No logarithms, no sigmoid!**

# Critic Loss: Maximize the Gap

## Critic's Objective

Maximize

$$[\mathbb{E}_{x \sim p_{\text{data}}} [C(x)] - \mathbb{E}_{z \sim p_z} [C(G(z))]]$$

or Minimize (Critic Loss Function)

$$\mathcal{L}_C = - [\mathbb{E}_{x \sim p_{\text{data}}} [C(x)] - \mathbb{E}_{z \sim p_z} [C(G(z))]]$$

## Intuition

- Critic tries to make **real scores high** and **fake scores low**
- Maximizes the **difference** between real and fake
- No upper/lower bound on scores (unlike  $[0, 1]$  probability)
- But Lipschitz constraint prevents infinite scores

# Generator Loss: Fooling the Critic

## Generator's Objective

Maximize

$$[-\mathbb{E}_{z \sim p_z}[C(G(z))]]$$

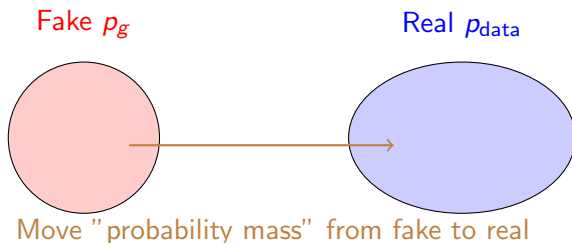
or equivalently Minimize (Generator Loss Function)

$$\mathcal{L}_G = [-\mathbb{E}_{z \sim p_z}[C(G(z))]]$$

## Intuition

- Generator wants fake images to get **high critic scores**
- Tries to **close the gap** between real and fake scores
- Makes fake distribution **indistinguishable** from real distribution

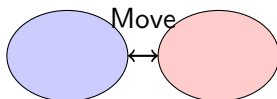
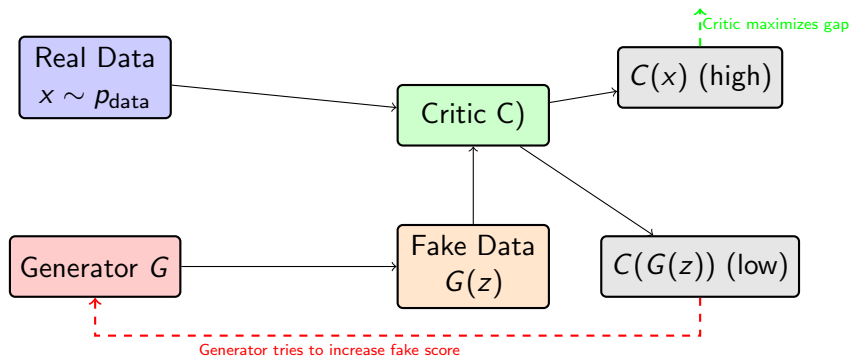
# Earth Mover's Distance: Moving Distributions



## Why "Earth Mover"?

- Imagine piles of dirt (probability mass) at fake distribution
- Need to move dirt to match real distribution
- **Work** = amount of dirt  $\times$  distance moved
- Critic learns the **optimal transport plan**

# WGAN Training: Moving the Distribution





# How WGAN Reduces Mode Collapse

## Why WGAN Encourages Diversity

Wasserstein loss penalizes the generator for **not covering all modes**



Collapsed mode: Same loss



Diverse modes: Lower Wasserstein distance

Wasserstein distance decreases when generator covers more modes

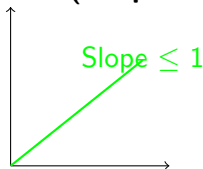
# The 1-Lipschitz Constraint

## What is Lipschitz Continuity?

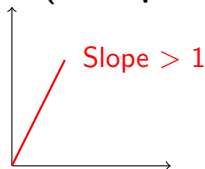
A function  $f$  is 1-Lipschitz if  $|f(x_1) - f(x_2)| \leq \|x_1 - x_2\|$

The function doesn't change too quickly.

### Good (1-Lipschitz)



### Bad (Not Lipschitz)



## How WGAN Enforces It

- Original WGAN: Weight clipping
- WGAN-GP: Gradient penalty in Loss Function

$$\lambda \mathbb{E}_{\hat{x}} [(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$$

# WGAN vs Standard GAN: Summary

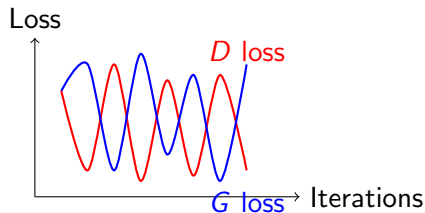
| Property                     | Standard GAN         | WGAN                      |
|------------------------------|----------------------|---------------------------|
| Loss function                | Log Loss             | Wasserstein Distance      |
| Discriminator output         | Probability $[0, 1]$ | Score $(-\infty, \infty)$ |
| Output activation            | Sigmoid              | Linear (no activation)    |
| Gradient saturation          | Yes                  | No                        |
| Mode collapse                | Common               | Rare                      |
| Training stability           | Unstable             | Stable                    |
| Loss correlates with quality | No                   | Yes                       |
| Lipschitz constraint         | Not needed           | Required                  |

## Key Takeaway

- WGAN provides **smooth, meaningful gradients** everywhere
- Generator **always receives feedback** on how to improve
- Training is **more stable** and **less sensitive** to hyperparameters
- Lower WGAN loss = better generation quality

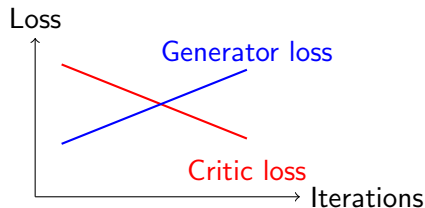
# Training Dynamics: GAN vs WGAN

## Standard GAN



Oscillating, unstable

## WGAN



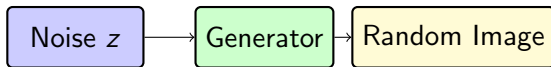
Smooth convergence

WGAN loss directly correlates with sample quality!

# The Problem: No Control Over Generation

## Standard GAN

- Input: Random noise  $z$
- Output: Random image
- **No way to control** what gets generated



No control over output

## Example

If you want a "dog" but get a "cat" → **Can't control it!**

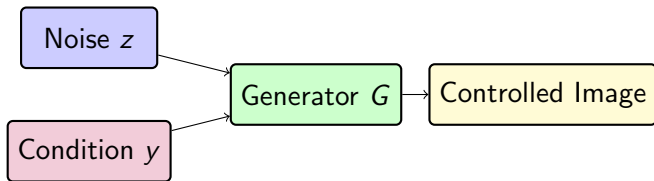
## Solution: Conditional GAN (cGAN)

Add **condition information**  $y$  to both generator and discriminator

# Conditional GAN: The Core Idea

## What is a Conditional GAN?

A GAN where both generator and discriminator receive **extra information**  $y$  (the condition)



Generator sees: noise + condition

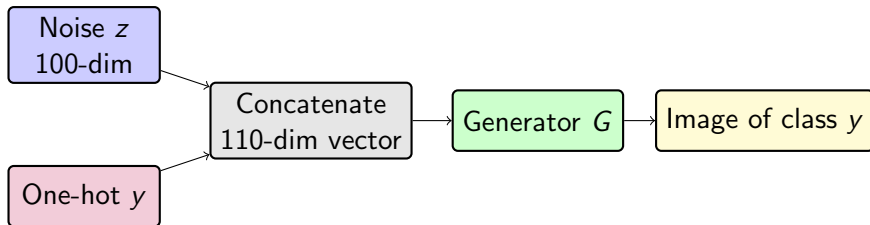
## Example

Condition  $y = \text{"dog"}$   $\rightarrow$  Generator creates a dog!

# Training Strategy: Generator Side

## How Generator Gets Condition

Condition  $y$  (e.g., class label) is **concatenated** with noise vector  $z$



Generator learns: given  $z$  and  $y$ , create image of class  $y$

## Example

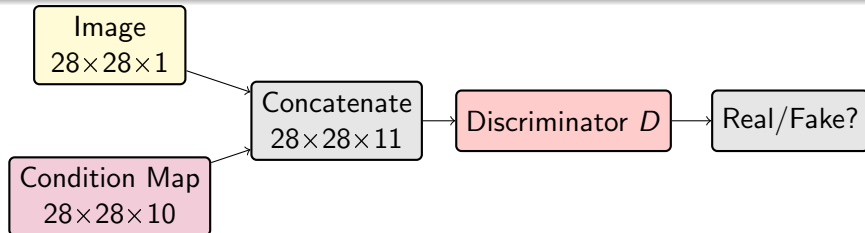
For MNIST (digits 0-9): One-hot vector has 10 dimensions

- $y = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$  means "generate a 3"

# Training Strategy: Discriminator Side

## How Discriminator Gets Condition

Condition  $y$  is **added as extra channels** to the input image



Discriminator checks: image AND condition match?

## Example

For MNIST: Condition map =  $28 \times 28$  grid where each pixel has the one-hot vector

- Tells discriminator: "This image should be a 3"
- Discriminator penalizes if image doesn't match condition



# Controlled Generation: Manipulating $z$

## How to Generate Diverse Outputs

Even with the same condition  $y$ , different noise  $z$  produces different variations

## Key Insight

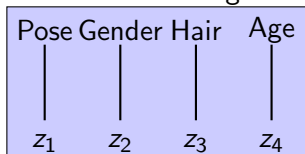
- $z$  controls **style, pose, writing style, expression**
- $y$  controls **class/category/content**
- Together: **full control** over generation

# The Problem: Latent Space Entanglement

## What is Entanglement?

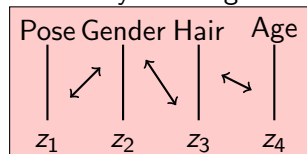
In standard GANs, dimensions of  $z$  are **mixed up** - one dimension controls multiple features

Ideal: Disentangled



Each dimension controls ONE feature

Reality: Entangled



Dimensions are mixed

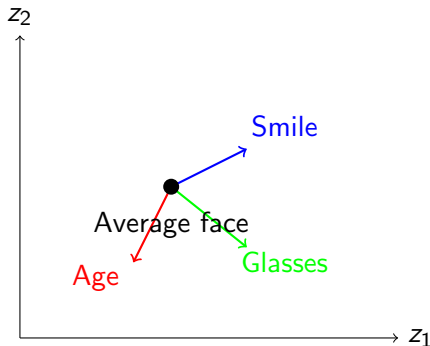
## Example of Entanglement

- Changing  $z_1$  might change BOTH smile AND hair color
- Cannot independently control features
- Makes fine-grained control difficult

# Finding Disentangled Directions in Latent Space

## Despite Entanglement, We Can Find Meaningful Directions

Even if  $z$  is entangled, some directions in latent space control specific features



Each arrow = moving in latent space changes ONE attribute

# Finding $z$ for a Desired Feature

## The Problem

We have a trained GAN. Given a **desired feature** (e.g., "glasses", "smile"), how do we find the noise vector  $z$  that generates it?

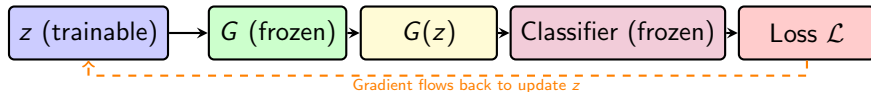
## The Solution

Instead of training the generator, we **optimize the input**  $z$  using gradient descent to maximize the feature score. We use a pre-trained feature classifier (e.g. "glasses" or "smile" classifier).

# Gradient-Based Optimization of $z$

## How It Works

- 1 Start with random  $z$
- 2 Generate image:  $x = G(z)$
- 3 Pass through feature classifier to get  $p = \text{Classifier}(x)$
- 4 Compute loss:  $\mathcal{L} = -\log(p)$  (want  $p \approx 1$ )
- 5 Backpropagate through classifier AND generator to update  $z$
- 6 Repeat until  $z$  produces desired feature



## Result

After optimization,  $z^*$  generates an image with the desired feature!

$$z^* = \arg \min_z \mathcal{L}_{\text{BCE}}(\text{Classifier}(G(z)), \text{target} = 1)$$

# Latent Space Arithmetic: The Magic Trick

## How to Find Features Directions in Z-Space

- Interpolate between images with/without attribute
- Subtract latent codes:  $z_{\text{smile}} = z_{\text{smiling}} - z_{\text{neutral}}$
- Add vector to any face to add smile!

## Vector Arithmetic in Latent Space

$$z_{\text{smiling woman}} = z_{\text{woman}} + z_{\text{smile}}$$

### Classic Example (Faces):

- $z_{\text{man}} - z_{\text{woman}} \approx$  "gender vector"
- Add to any face to change gender
- Works surprisingly well!

### Other Examples:

- $z_{\text{old}} = z_{\text{old}} - z_{\text{young}}$
- $z_{\text{old woman}} = z_{\text{woman}} + z_{\text{old}}$

# Applications of Conditional GANs

## Image-to-Image Translation

- Sketch  $\rightarrow$  Photo
- Day  $\rightarrow$  Night
- Black & White  $\rightarrow$  Color
- Satellite  $\rightarrow$  Map

## Text-to-Image

- "A red bird on a branch"
- "A cat wearing a hat"



## Attribute Editing

- Add/remove glasses
- Change hair color
- Make older/younger
- Change expression

## Data Augmentation

- Generate rare classes
- Balance imbalanced datasets



# Summary: Conditional GANs

## Key Takeaways

- **Condition**  $y$  controls what gets generated
- Generator: concatenate  $y$  with noise  $z$
- Discriminator: add  $y$  as extra channels to input
- Enables **controlled, diverse** generation
- Latent space is **entangled** but we can find meaningful directions
- **Vector arithmetic** enables attribute manipulation



# References



I. Goodfellow et al. (2014).  
Generative Adversarial Networks.  
*NIPS*.



A. Radford, L. Metz, S. Chintala (2015).  
Unsupervised Representation Learning with Deep Convolutional GANs.  
*ICLR*.



M. Arjovsky, S. Chintala, L. Bottou (2017).  
Wasserstein GAN.  
*ICML*.



I. Gulrajani et al. (2017).  
Improved Training of Wasserstein GANs.  
*NIPS*.



T. Karras, S. Laine, T. Aila (2019).  
A Style-Based Generator Architecture for GANs.  
*CVPR*.